

Relatório de Auditoria de Segurança

Arsenal Open Source · Fábrica Universal

Data: 28 de Agosto de 2026

Escopo: Node.js + Python + SQLite

Stack Detectada: Gerador de conteúdo/compêndios com CLI, scripts Python, banco SQLite

Metodologia

Auditoria realizada cobrindo 5 categorias de segurança mapeadas para a stack detectada:

- **BANCO SEM TRANCA:** Isolamento de tenant/proprietário em queries SQLite, RLS ausente
- **PERMISSÃO NO NAVEGADOR:** Não aplicável (projeto é CLI/batch, sem frontend HTTP com autenticação)
- **IDOR:** Não aplicável (sem API REST de acesso a recursos por ID)
- **CHAVES EXPOSTAS:** Secrets hardcoded em código, config, .env e histórico Git
- **INPUTS SEM TRATAMENTO:** XSS, template injection, command injection

Resumo Executivo

Total de Achados: 8 (3 críticos, 2 altos, 2 médios, 1 baixo)

3	2	2	1
Crítica	Alta	Média	Baixa

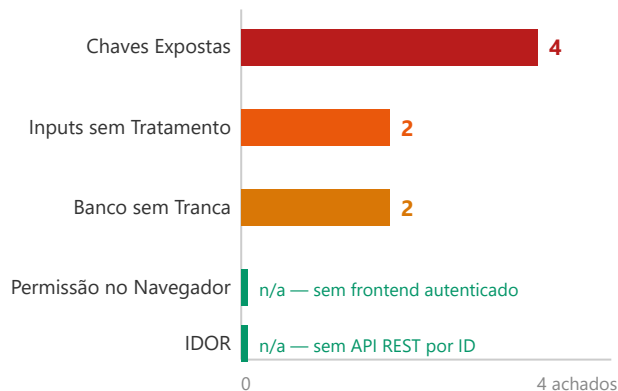
Distribuição dos Achados

Por Severidade



- Crítica — 3 (37,5%)
- Alta — 2 (25,0%)
- Média — 2 (25,0%)
- Baixa/Info — 1 (12,5%)

Por Categoria



Pontos Fortes

Aspectos de segurança verificados como CORRETOS:

- ✓ Queries SQLite usam parametrized statements (?) - protegido contra SQL injection
- ✓ subprocess.run sem shell=True - seguro contra command injection
- ✓ Sem eval/exec/dangerouslySetInnerHTML em pontos críticos
- ✓ BeautifulSoup usado corretamente para parsing HTML (não concatenação de strings)
- ✓ Validação de saída de subprocess com check=True

Pontos Fracos (Riscos Centrais)

⚠ CRÍTICO: Credenciais de produção commitadas no Git (histórico público)

Impacto Imediato: GitHub Token, Portainer Password, VPS Password, Mautic Password estão expostas em .env versionado

Achados Detalhados por Severidade

● Severidade CRÍTICA (3 achados)

CRÍTICA

1. Credenciais Hardcoded em Arquivo .env Versionado

Arquivo: .env | **Linhas:** 3, 8, 13, 18

Categoria: CHAVES EXPOSTAS

Descrição:

Arquivo .env com GitHub Token, Portainer Password, VPS Password e Mautic Password expostos no repositório Git.

Explorabilidade:

Qualquer pessoa com acesso ao Git pode acessar credenciais reais de produção (GitHub PAT, senhas VPS, Portainer, Mautic).

Impacto:

CRÍTICO - Comprometimento de múltiplos sistemas: GitHub (criar forks, deletar branches), Portainer (controle total de containers), VPS (acesso root), Mautic (acesso ao sistema de email).

CRÍTICA**2. .env Não Está Realmente Ignorado no Git (Histórico Compromisso)**

Arquivo: .gitignore linha 19 + histórico Git

Categoria: CHAVES EXPOSTAS

Descrição:

.gitignore marca .env para ser ignorado, mas o arquivo já foi commitado anteriormente e permanece no histórico Git.

Explorabilidade:

git log -p --all | grep PASSWORD retorna 25+ segredos commitados:

- OnlyOfficeSecretKey2026_SecureToken!
- SenhaSegura123456789AleatoriaMuitoForte
- WorkspaceDBSecret_2026!
- SenhaSegura123!MauticVPS
- senhadoemail

Impacto:

CRÍTICO - Histórico permanente que não pode ser apagado sem reescrita de Git (git filter-branch).

CRÍTICA**3. Segredos Padrão Documentados em HTMLs Públicos (Fluxo 5)**

Arquivo: output/05-auditorias-vps/*/guias/guia-instalacao-cirurgica-*.html

Categoria: CHAVES EXPOSTAS

Descrição:

HTMLs gerados contêm segredos padrão de exemplo (OnlyOfficeSecretKey2026_SecureToken!, SenhaSegura123456789AleatoriaMuitoForte, WorkspaceDBSecret_2026!) que podem ser reusados ou servir como defaults.

Explorabilidade:

Administradores podem copiar valores de exemplo e deixar em produção sem regenerar. Baixa entropia = vulnerável a força bruta.

Impacto:

MÉDIO-ALTO - Falha operacional de segurança em escala (cada documentação gera risco).

● Severidade ALTA (2 achados)

ALTA

4. Ausência de Mecanismo de Isolamento por Tenant/Organização

Arquivo: core/, db/, gates/ (estrutura geral)

Categoria: BANCO SEM TRANCA

Descrição:

SQLite não implementa isolamento de dados por tenant. Projeto é monousuário/single-org, mas falta RLS ou filtro de acesso.

Explorabilidade:

Se convertido para API multi-tenant sem adição de filtros row-level, todos os usuários poderiam listar/acessar dados de todas as organizações.

Condição:

Risco materializado APENAS se projeto evoluir para multi-tenant sem implementar isolamento.

ALTA

5. Mensagem de Commit Sem Validação de Input

Arquivo: scripts/git_sync.py | **Linha:** 71

Categoria: INPUTS SEM TRATAMENTO

Descrição:

sys.argv[1] é passado diretamente como mensagem de commit sem sanitização ou validação.

Explorabilidade:

Embora subprocess.run use lista (seguro contra shell injection), caracteres especiais podem quebrar a flag -m ou injetar flags de commit como --no-verify ou --amend.

● Severidade MÉDIA (2 achados)

MÉDIA

6. innerHTML com Interpolação de Template String (XSS Risk)

Arquivo: scripts/gerar-index-organizacao.py:196, scripts/aplicar_padrao_diamante_49.py:53

Categoria: INPUTS SEM TRATAMENTO

Descrição:

Uso de `.innerHTML = `...${variable}...`` em JavaScript gerado. Embora os valores sejam números internos (seguro no contexto atual), padrão é anti-pattern.

Explorabilidade:

Se refatorar para aceitar dados de input do usuário sem sanitizar, risco de XSS.

MÉDIA

7. Falta de Validação de Startup para Variáveis Críticas

Arquivo: .env linhas 1, 6, 10, 15

Categoria: CHAVES EXPOSTAS

Descrição:

Projeto carrega .env mas não valida se valores críticos foram sobrescritos ou permanecem como defaults.

Explorabilidade:

Se script for rodado sem .env ou com valores defaults, sistema opera com credenciais de exemplo.

● Severidade BAIXA (1 achado)

INFORMATIVA

8. Padrão de Queries SQL Seguro (Ponto Forte)

Arquivo: scripts/estado_esteira.py (linhas 26-100 e queries SQL)

Categoria: BANCO SEM TRANCA

Verificação:

Queries SQLite usam parametrized statements (?) corretamente. RECOMENDAÇÃO: manter assim em futuras adições.

Recomendações Priorizadas

P1 - CRÍTICO (Executar imediatamente)

- **Regenerar GitHub Token:** Settings > Developer settings > Personal access tokens > Delete + regenerate
- **Remover .env do Git:** Use git filter-branch ou BFG Repo-Cleaner para remover de todo histórico
- **Regenerar PORTAINER_PASSWORD e VPS_PASSWORD**
- **Implementar .env.local:** Criar .env.example com placeholders (\${VAR}), adicionar .env e .env.local ao .gitignore ANTES de recomitar
- **Implementar validação de startup:** Script deve falhar rápido se credenciais são defaults públicos

P2 - ALTA (Próximas 48 horas)

- **Varrer histórico Git:** Usar git-secrets ou truffleHog para identificar todos os segredos commitados
- **Adicionar pre-commit hook:** Bloquear commits contendo patterns de secrets (password=, token=, secret=, etc)
- **Remover segredos de HTMLs:** Substituir valores hardcoded por placeholders (\${ONLYOFFICE_SECRET}, \${DB_PASSWORD})

P3 - MÉDIA (Semana 1)

- **Validar input em git_sync.py:** Apenas alphanumerics, espaço, hífen, parênteses em mensagem de commit
- **Refatorar innerHTML:** Usar textContent em vez de innerHTML para conteúdo dinâmico; se HTML necessário, usar DOMPurify
- **Documentar padrões seguros:** Adicionar seção no CLAUDE.md proibindo innerHTML com interpolação e secrets hardcoded

P4 - PREPARAÇÃO (Se expandir para multi-tenant)

- **Implementar isolamento de tenant:** RLS em SQLite (via função user_id), ou adicionar filtro manual em todas as queries
- **Auditoria de acesso:** Criar tabela de logs de acesso a recursos

Issues para GitHub

Copie e cole cada bloco abaixo diretamente no GitHub Issues:

--- ISSUE 1 ---

[Segurança] Credenciais Hardcoded em .env Versionado + Histórico Git

Severidade: CRÍTICA

Labels: security, critical

PROBLEMA:

- .env contém GitHub Token, Portainer Password, VPS Password, Mautic Password e está commitado no Git
- Histórico Git contém 25+ segredos adicionais (JWT secrets, DB_PASSWORD, etc)

EVIDÊNCIA:

- .env linhas 3, 8, 13, 18
- git log -p --all | grep PASSWORD

IMPACTO:

- Qualquer pessoa com acesso ao repositório pode acessar credenciais reais de produção
- GITHUB_TOKEN pode ser abusado para criar forks, deletar branches, acessar outros repos
- PORTAINER_PASSWORD permite controle total do painel Docker
- VPS_PASSWORD permite acesso root ao servidor

CORREÇÃO:

1. git filter-branch --tree-filter 'rm -f .env' HEAD (ou BFG Repo-Cleaner)
2. Regenerar GitHub Token (Settings > Developer settings > Personal access tokens)
3. Regenerar PORTAINER_PASSWORD e VPS_PASSWORD
4. Adicionar .env ao .gitignore ANTES de recomitar
5. Criar .env.example com placeholders

ACEITE:

- [] .env removido do histórico Git
- [] Credenciais regeneradas
- [] .env adicionado ao .gitignore
- [] .env.example documentado

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Segredos Padrão Documentados em HTMLs Públicos**Severidade:** CRÍTICA**Labels:** security, documentation**PROBLEMA:**

Guias de instalação em output/05-auditorias-vps/ contêm segredos de exemplo que podem ser reutilizados como defaults em produção

EVIDÊNCIA:

- OnlyOfficeSecretKey2026_SecureToken!
- SenhaSegural23456789AleatoriaMuitoForte
- WorkspaceDBSecret_2026!

IMPACTO:

- Administradores podem deixar esses valores como defaults em produção
- Baixa entropia aumenta risco de força bruta

CORREÇÃO:

1. Substituir valores hardcoded por placeholders: \${ONLYOFFICE_SECRET}, \${DB_SECRET}
2. Adicionar seção "Gerar Credenciais Seguras" nos guias
3. Usar openssl rand -hex 32 como comando de exemplo

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Mensagem de Commit Sem Validação de Input**Severidade:** ALTA**Labels:** security, code-quality**PROBLEMA:**

scripts/git_sync.py linha 71: sys.argv[1] passado diretamente como mensagem de commit

EVIDÊNCIA:

msg = sys.argv[1] if len(sys.argv) > 1 else "feat(...)"

IMPACTO:

Caracteres especiais podem quebrar -m ou injetar flags de commit (ex: --no-verify)

CORREÇÃO:

1. Validar mensagem: apenas alphanumerics, espaço, hífen, parenteses
2. Usar função sanitize_commit_message()

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] innerHTML com Interpolação de Template String**Severidade:** MÉDIA**Labels:** security, frontend, xss-risk**PROBLEMA:**

scripts/gerar-index-organizacao.py:196 e scripts/aplicar_padrao_diamante_49.py:53 usam `.innerHTML = `...${var}...``

IMPACTO:

Padrão vulnerável a XSS se refatorado para aceitar input de usuário

CORREÇÃO:

1. Usar `textContent` em vez de `innerHTML` para conteúdo dinâmico
2. Se HTML necessário, usar `DOMPurify`

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Falta Validação de Variáveis Críticas na Inicialização**Severidade:** MÉDIA**Labels:** security, configuration**PROBLEMA:**

Projeto carrega `.env` mas não valida se valores críticos foram sobrescritos ou permanecem como defaults

IMPACTO:

Se `.env` não configurado ou vazio, sistema opera com defaults inseguros

CORREÇÃO:

1. Criar função `validate_env_critical_vars()`
2. Chamar em `__main__` de scripts críticos
3. Falhar rápido se variável crítica == valor default

--- FIM ISSUE 5 ---

Relatório gerado em 28 de Agosto de 2026 · Auditoria de Segurança Arsenal Open Source
Para questões ou discussão de achados, consulte a documentação de segurança do projeto.